United International University
Department of Computer Science and Engineering
CSE 4811 / DS 4213: Natural Language Processing

Fall 2026

Practice Problems: Class 04

---

1. **Word vectors.**

   (a) True or false: one-hot vector encodings of words must be the length of the entire vocabulary, while learned embeddings are generally much smaller.

   (b) The input embedding table E and the output layer use separate parameters. The table is used only for lookup. One backward pass looks up IDs $[2, 5, 2]$; the loss gradients with respect to the three returned vectors are $[2, 1]$, $[0, 3]$ and $[-1, 4]$, in that order. These are the only uses of the table. Write the two-component gradient of the loss for row 2 of E.

   UW *CSE 493G1 Deep Learning* **2024** Quiz 3, Q1.1 · Solutions

   Johns Hopkins *601.471 NLP: Self-supervised Models* **2026** Quiz 4, Q1(b) · Solutions

2. **Counting parameters.** Consider a feedforward language model. In this model, the probability $P(w_i \mid w_{i-n+1}, \ldots, w_{i-1})$ is given by:

$$
\begin{aligned}
P(w_i \mid w_{i-n+1}, \ldots, w_{i-1}) &= \text{softmax}(\mathbf{U}\mathbf{h}_2 + \mathbf{b}_2) \\
\mathbf{h}_2 &= \tanh(\mathbf{W}\mathbf{h}_1 + \mathbf{b}_1) \\
\mathbf{h}_1 &= \text{concatenate}(\mathbf{C}\mathbf{w}_{i-n+1}, \ldots, \mathbf{C}\mathbf{w}_{i-1}) \\
\mathbf{w}_i &= \text{onehot}(w_i) \quad \text{for all } i
\end{aligned}
$$

   In this notation, $\mathbf{U}$, $\mathbf{W}$ and $\mathbf{C}$ are matrices, while $\mathbf{b}_1$, $\mathbf{b}_2$, $\mathbf{h}_1$, $\mathbf{h}_2$ and $\mathbf{w}_{i-n+1}, \ldots, \mathbf{w}_{i-1}$ are vectors. The parameters of the model are $\mathbf{U}$, $\mathbf{W}$, $\mathbf{C}$, $\mathbf{b}_1$ and $\mathbf{b}_2$, while the remaining variables are intermediate layers computed by the network. Now consider the number of parameters required to represent this model. This number is determined by the size of the vocabulary $|V|$ (given to you by the data), the order n, and the dimension of the two hidden layers, $\mathbf{h}_1$ and $\mathbf{h}_2$, which we will denote d_1 and d_2, respectively (note that the first dimension must be divisible by $n-1$, but you can ignore this detail in your calculations).

   (a) How would you express the number of model parameters in terms of $|V|$, n, d_1 and d_2?

   (b) An effective size for the hidden dimension of a neural NLP model is often in the hundreds. Let $|V| = 20{,}000$. For n from 2 to 5, how many parameters would your model have if $d_1/(n-1) = d_2 = 100$? What if $d_1/(n-1) = d_2 = 1000$?

   (c) What do you conclude about the relative memory efficiency of classic n-gram and feedforward neural language models? If you increased n even further, what would happen?

   Edinburgh *Natural Language Understanding, Generation, and Machine Translation* **2025** Tutorial 1, Q2 · Solutions

3. **Counting and learning.**

   (a) Consider how a neural language model with a softmax output layer compares with a classic n-gram language model. Typically, we use techniques like smoothing or backoff in conjunction with n-gram models. Does the problem these techniques solve arise in the neural model? Why or why not?

   (b) Recall the neural language model architecture: the representations of the three previous words, rows of E, are joined into one vector e; the hidden units are $h = \sigma(We+b)$; and $\hat{y} = \text{softmax}(Uh)$ is the distribution for the next word. Why is it that the learned word representations for "professor" and "teach" are likely to be far apart? (It is not sufficient to say they are "semantically dissimilar"; your answer should mention how the representations are used in the prediction task.)

(c) Two prefixes differ in an earlier name but have the same final four tokens, `the key belonged to`. A deterministic fixed-window language model sees only those four tokens. With the same model parameters used for both prefixes, can adding hidden layers alone make their next-token distributions differ? Answer yes or no, explain why in one sentence, and name one input change that would expose the distinction.

Edinburgh *Natural Language Understanding, Generation, and Machine Translation* **2025** Tutorial 1, Q1(d) · Solutions

Toronto *CSC321 Introduction to Neural Networks and Machine Learning* **2017** Final, Q5 · Solutions

Johns Hopkins *601.471 NLP: Self-supervised Models* **2026** Quiz 4, Q1(c) · Solutions

4. **Skip-gram and CBOW.**

(a) Select the answer that includes all the skip-gram (word, context) training pairs for the sentence *the cat ran away*, for a window size ±2 from the target.

A. [the, cat], [the, ran], [cat, ran], [cat, away], [ran, away]
B. [the], [cat], [ran], [away]
C. [the, cat], [the, ran], [cat, the], [cat, ran], [cat, away], [ran, the], [ran, cat], [ran, away], [away, cat], [away, ran]
D. [the, ran], [ran, cat], [cat, away]
E. [the, cat ran], [cat, the], [cat, ran away], [ran, the cat], [ran, away], [away, cat ran]

(b) In one or two sentences, briefly explain the difference between the skip-gram and continuous bag-of-words (CBOW) methods.

UT Austin *CS388 Natural Language Processing* **2022** Midterm, Q1(1) and Q2(a) · Solutions

5. **The cost of the softmax.** A feedforward neural network language model (NNLM) can be used as another architecture for training word vectors. This model tries to predict a word given the N words that precede it. To do so, we concatenate the word vectors of N previous words and send them through a single hidden layer of size H with a tanh nonlinearity and use a softmax layer to make a prediction of the current word. The size of the vocabulary is V. The model is trained using a cross entropy loss for the current word. Let the word vectors of the N previous words be $x_1, x_2, \ldots, x_N$, each a column vector of dimension D, and let y be the one-hot vector for the current word. The network is specified by the equations below:

$$x = [x_1; x_2; \ldots; x_N], \qquad h = \tanh(Wx + b), \qquad \hat{y} = \text{softmax}(Uh + d), \qquad J = -\sum_i y_i \log \hat{y}_i,$$

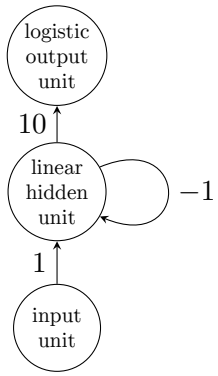
with $x \in \mathbb{R}^{ND}$, $W \in \mathbb{R}^{H \times ND}$, $b \in \mathbb{R}^H$, $h \in \mathbb{R}^H$, $U \in \mathbb{R}^{V \times H}$, $d \in \mathbb{R}^V$ and $\hat{y} \in \mathbb{R}^V$.

(a) What is the complexity of forward propagation in an NNLM for a single training example?

(b) Describe at least one way to change the model that would reduce this complexity.

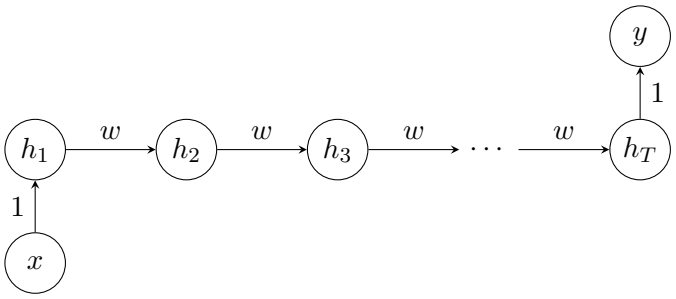
Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 3, Q3(b) · Solutions

6. **A small RNN.** Determine what the following recurrent network computes. More precisely, determine the function computed by the output unit at the final time step; the other outputs are not important. All of the biases are 0. You may assume the inputs are integer valued and the length of the input sequence is even.



Toronto *CSC321 Introduction to Neural Networks and Machine Learning* **2017** Final, Q10 · Solutions

7. **Gradients through time.** Consider the following RNN, which has a scalar input at the first time step, makes a scalar prediction at the last time step, and uses a shifted logistic activation function: $\phi(z) = \sigma(z) - 0.5$.



For $t < T$, backpropagation gives $\partial L/\partial h_t = w\,\sigma'(z_{t+1})\,\partial L/\partial h_{t+1}$, where L is the loss computed from the prediction y and z_t is the input to the activation function at time t.

(a) Suppose the input to the network is $x = 0$. Notice that $h_t = 0$ for all t. Based on the formula above, determine the value α such that if $w < \alpha$, the gradient vanishes, while if $w > \alpha$, the gradient explodes. You may use the fact that $\sigma'(0) = 1/4$.

(b) True or false: clipping the gradient (cutting off at a threshold) will solve the exploding gradients problem. Briefly justify your answer.

Toronto *CSC321 Introduction to Neural Networks and Machine Learning* **2017** Final, Q12 · Solutions

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 2, Q4.1(d) · Solutions

8. **Running an RNN language model.** An RNN language model computes its state by the recursion $h_t = f(h_{t-1}, w_t)$, starting from a fixed state h_0 of d numbers. An output matrix V, with one row for each word of the vocabulary and one for `</s>`, gives the next-word probabilities $p(w_t \mid w_1 \dots w_{t-1}) = \text{softmax}(Vh_{t-1})_{w_t}$. Now answer the following questions in terms of the length n of the text.

(a) What is the runtime of computing the probability $p(w_t \mid w_1 \dots w_{t-1})$?

(b) What is the runtime of computing the probability $p(w_1 \dots w_n)$?

(c) Considering the answers to the previous two questions, what does the common claim that "RNN language models are non-parallelizable" mean? How does that inhibit training on large corpora?

(d) How many computational steps are between the incorporation of w_1 and the computation of $p(w_t \mid w_1 \dots w_{t-1})$, given that we can fully parallelize the computation?

ETH Zurich *Large Language Models* **2024** Week 3 exercises, Q1, parts 1 to 4 · Solutions

9. **LSTM gates.** Here are the defining equations for an LSTM cell.

$$i_t = \sigma\big(W^{(i)}x_t + U^{(i)}h_{t-1}\big), \qquad f_t = \sigma\big(W^{(f)}x_t + U^{(f)}h_{t-1}\big), \qquad o_t = \sigma\big(W^{(o)}x_t + U^{(o)}h_{t-1}\big),$$
$$\tilde{c}_t = \tanh\big(W^{(c)}x_t + U^{(c)}h_{t-1}\big), \qquad c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t, \qquad h_t = o_t \circ \tanh(c_t).$$

Recall that $\circ$ denotes element-wise multiplication and that σ denotes the sigmoid function. True or false? Briefly justify each answer.

(a) If x_t is the 0 vector, then $h_t = h_{t-1}$.

(b) If f_t is very small or zero, then error will not be back-propagated to earlier time steps.

(c) The entries of f_t, i_t, o_t are non-negative.

(d) f_t, i_t, o_t can be viewed as probability distributions (that is, their entries are non-negative and their entries sum to 1).

Stanford *CS224N Natural Language Processing with Deep Learning* **2018** Practice midterm 2, Q4.2 · Solutions

10. **Gradients in an LSTM.** Select all that apply:

    A. One motivation of LSTMs is to reduce the vanishing gradient problem seen with RNNs.
    B. It is not possible for LSTMs to experience the vanishing gradient problem.
    C. It is possible for LSTMs to experience the vanishing gradient problem.
    D. It is not possible for LSTMs to experience the exploding gradient problem.
    E. It is possible for LSTMs to experience the exploding gradient problem.

CMU *10-423 Generative AI* **2025** Practice exam, Q1.4 · Solutions